

DSA Project Final Submission

Showroom Management System

Course: Data Structures and Algorithms (DSA)

Group Members

1. Abdullah Asif (231370148)
2. Zohair Shazib (231370164)

Instructor Name Dr Qamar Aksari

Abstract: Showroom Management System

The **Showroom Management System** is a Java-based application designed to efficiently manage a car showroom using **custom data structures**. It enables **customers** to view, search, and purchase cars, **employees** to manage sales and profiles, and **managers** to oversee inventory, employees, and pricing.

Key data structures include **Hash Maps for quick lookups, BSTs for sorting, Linked Lists for purchase history, and Queues for order management**. The system ensures efficiency with **exception handling** and a structured approach to inventory and transactions.

Project Objectives

The objectives of this project are as follows:

To design a **Showroom Management System** that:

- Allows customers to view available cars and their details.
- Facilitates the purchase of cars from the showroom inventory.
- Tracks purchased cars and displays a customer's purchase history.
- Updates the car inventory and adds/removes cars.

To implement Custom Data Structures to manage the showroom's car inventory, customer data, and purchase history efficiently.

The system will utilize a combination of the following data structures:

- Array/List for storing car details.
- Hash Map for fast access to car details using the CarID.
- Binary Search Tree (BST) for sorting and searching cars based on attributes like
- price or model.
- Linked List for tracking customer purchases.

Technologies and Tools

- Programming Language: Java
- Data Structures: Custom implementations of arrays, hash maps, binary search trees, linked lists, and queues. These will be used to store and manipulate the car inventory, customer data, and purchase history efficiently.
- Exception Handling: Java's built-in Exception Handling (try, catch, throw) will be utilized to handle errors and ensure the program remains robust, even when faced with invalid inputs or unexpected conditions.
- User Interface: The system will feature a basic text-based interface (CLI) for interacting with users. For simplicity and efficiency, the interface will be menu-driven, allowing the user to easily access functionalities like viewing available cars, making purchases, and checking purchase history.

Key Features

The Showroom Management System will provide the following key features:

View Available Cars

- Functionality: Users can view a list of all cars available in the showroom, displaying details like CarID, Model, Brand, Price, Year, and Availability.
- Data Structure Used: Array/List will store all available cars. Sorting and filtering will be handled by the Binary Search Tree (BST) for attributes such a Price, Brand, and Year.

Search Cars by Attributes

- Functionality: Users can search for cars by different attributes, including Price Range, Brand, Model, or Year.
- Data Structure Used: Binary Search Tree (BST) will enable efficient searching and sorting of cars based on the selected attribute. The Hash Map will be used to quickly access and display detailed information of specific cars.

Purchase a Car

- Functionality: Customers can purchase a car from the showroom, after which the car will be removed from the available inventory.
- Data Structure Used: Queue will manage the purchase requests in FIFO (First In, First Out) order. The Linked List will store each customer's purchase history, including car details and the purchase date.

View Purchased Cars

- Functionality: Customers can view a list of cars they have purchased, along with the purchase details.
- Data Structure Used: Linked List will store the purchase history for each customer, where each node represents a car purchased by the customer. View Detailed Information of a Car
- Functionality: By selecting a car from the available inventory, customers can view detailed information like Price, Model, Brand, Year, and Availability.
- Data Structure Used: Hash Map will be used to quickly access detailed car information using the CarID as the key.

Manage Car Inventory

- Functionality: Admin users can add new cars to the showroom inventory or remove cars that have been sold.
- Data Structure Used: Array/List will be used to maintain the inventory list. Updates to the inventory will be reflected by adding or removing cars from the list.

Sort Cars by Price or Year

- Functionality: Users can sort cars based on different criteria like Price or Year.
- Data Structure Used: Binary Search Tree (BST) will be used to keep the cars sorted automatically. Alternatively, Array/List could be sorted when needed.

Handle Exception Scenarios

- Functionality: The system will have built-in exception handling for scenarios like invalid user inputs, unavailable cars, or unsuccessful purchases.
- Technology Used: Java's Exception Handling (try, catch, throw) will ensure that the system remains robust and continues to function smoothly even when unexpected errors occur (e.g., trying to purchase a car that is already sold).

Customer Profile and Purchase History

- Functionality: Each customer will have a profile to track their purchases. They can view their purchase history and any other personal details (e.g., name, contact details).
- Data Structure Used: Linked List will store each customer's purchase history.

Data Structures Used (Analysis)

- **Doubly Linked List** → Used for managing purchase history, employee list
- **Hash Map** → Used for fast retrieval of cars using CarID
- **Binary Search Tree (BST)** → Used for sorting cars based on price/model
- **Queue** → Used for managing customer purchases in FIFO order
- **ArrayList** → Used for storing invoices and managing dynamic collections that require frequent access.

UML Relationships with Cardinalities

(1) Showroom ◆— (many) Cars

(1) Showroom ◆— (many) Employees

(1) Showroom ◆— (many) Invoices

(1) Customer ◆— (many) Owned Cars

(1) Customer ◆— (many) Purchase History

(1) Invoice ◆— (1) Payment

(1) Cart ◆— (many) Cars

(1) Person ▲— (1) Employee

(1) Employee ▲— (1) Manager

(1) Person ▲— (1) Customer

(1) UsingShowroom ◇— (1) Showroom

(1) UsingShowroom ◇— (many) Customers

(1) UsingShowroom ◇— (many) Employees

(1) UsingShowroom ◇— (many) Managers

UML ShowRoom Management System

Customer

- int : customerId
- String : username
- String : password
- DoublyLinkedList<Car> : ownedCars
- DoublyLinkedList<Invoice> : purchaseHistory

+ Customer()
+ Customer(String name, long phoneNumber, int customerId, String username, String password)
+ void : setCustomerId(int customerId)
+ void : setUsername(String username)
+ void : setPassword(String password)
+ int : getCustomerId()
+ String : getUsername()
+ String : getPassword()
+ DoublyLinkedList<Car> : getOwnedCars()
+ void : addCar(Car car)
+ void : addPurchaseHistory(Invoice invoice)
+ DoublyLinkedList<Invoice> : getPurchaseHistory()
+ void : display()

Person

- String : name
- long : phone

+ Person()
+ Person(String name, long phone)
+ void : setName(String name)
+ void : setPhone(long phone)
+ String : getName()
+ long : getPhone()
+ void : display()

Manager

- String : department

+ Manager(String name, long phone, int empId, String username, String password, double salary, String department)
+ void : setDepartment(String department)
+ String : getDepartment()
+ void : setPassword(String password)
+ void : display()
+ String : toString()

Car

- int : carId
- String : make
- String : model
- double : price
- Engine : engine

+ Car(int carId, String make, String model, double price, String engineType, int horsepower)
+ void : setCarId(int carId)
+ void : setMake(String make)
+ void : setModel(String model)
+ void : setPrice(double price)
+ int : getCarId()
+ String : getMake()
+ String : getModel()
+ double : getPrice()
+ Engine : getEngine()
+ int : compareTo(Car other)
+ void : display()
+ String : toString()
+ boolean : equals(Object obj)
+ int : hashCode()

Employee

- int : empId
- String : username
- String : password
- double : salary

+ Employee(String name, long phone, int empId, String username, String password, double salary)
+ void : setEmpId(int empId)
+ void : setUsername(String username)
+ void : setPassword(String password)
+ void : setSalary(double salary)
+ int : getEmpId()
+ String : getUsername()
+ String : getPassword()
+ double : getSalary()
+ void : display()
+ String : toString()

Invoice

- String : customerName
- DoublyLinkedList<Car> : purchasedCars
- double : totalAmount
- Payment : paymentDetails

+ Invoice(String customerName, DoublyLinkedList<Car> purchasedCars, double totalAmount, Payment paymentDetails)
+ String : getCustomerName()
+ DoublyLinkedList<Car> : getPurchasedCars()
+ double : getTotalAmount()
+ Payment : getPaymentDetails()
+ String : toString()

Payment

- String : cardNumber
- String : cardHolderName
- double : amount

+ Payment(String cardNumber, String cardHolderName, double amount)
+ void : setCardNumber(String cardNumber)
+ void : setCardHolderName(String cardHolderName)
+ void : setAmount(double amount)
+ String : getCardNumber()
+ String : getCardHolderName()
+ double : getAmount()
+ String : maskCardNumber(String cardNumber)
+ String : toString()

Engine

- String : type
- int : horsepower

+ Engine(String type, int horsepower)
+ void : setType(String type)
+ void : setHorsepower(int horsepower)
+ String : getType()
+ int : getHorsepower()
+ void : display()
+ String : toString()

Cart

- DoublyLinkedList<Car> : items

+ Cart()
+ void : addToCart(Car car)
+ void : removeFromCart(Car car)
+ void : clearCart()
+ boolean : isEmpty()
+ DoublyLinkedList<Car> : getItems()
+ double : calculateTotal()
+ void : displayCart()

Showroom

- String : name
- String : location
- HashMap<Integer, Car> : cars
- DoublyLinkedList<Employee> : employees
- AVLTree<Car> : sortedCars
- ArrayList<Invoice> : invoices

+ Showroom(String name, String location)
+ void : addCar(Car car)
+ boolean : removeCar(int carId)
+ boolean : updateCarPrice(int carId, double newPrice)
+ void : addEmployee(Employee employee)
+ boolean : removeEmployee(int empId)
+ boolean : updateEmployeeSalary(int empId, double newSalary)
+ void : addInvoice(Invoice invoice)
+ void : printInvoices()
+ void : display()
+ void : printEmployees()
+ void : printSortedCars()
+ Iterator<Employee> : iterator()

UsingShowroom

(No attributes - only static methods)

+ void : main(String[] args)
+ void : logInUser(Scanner scan, DoublyLinkedList<Manager> managers, DoublyLinkedList<Employee> employees, DoublyLinkedList<Customer> customers, Showroom showroom)
+ boolean : managerLogin(Scanner scan, DoublyLinkedList<Manager> managers, Showroom showroom, String username, String password)
+ boolean : employeeLogin(Scanner scan, DoublyLinkedList<Employee> employees, Showroom showroom, String username, String password)
+ boolean : customerLogin(Scanner scan, DoublyLinkedList<Customer> customers, Showroom showroom, String username, String password)
+ void : customerOperations(Scanner scan, Showroom showroom, Customer customer)
+ void : employeeOperations(Scanner scan, Showroom showroom, Employee loggedInEmployee)
+ void : managerOperations(Scanner scan, Showroom showroom, Manager manager)
+ void : updateCustomerProfile(Scanner scan, Customer customer)
+ void : updateEmployeeProfile(Scanner scan, Employee employee)
+ void : updateManagerProfile(Scanner scan, Manager manager)
+ void : viewPurchaseHistory(Customer customer)
+ void : signUpCustomer(Scanner scan, DoublyLinkedList<Customer> customers)
+ void : addEmployee(Scanner scan, Showroom showroom)
+ void : addCar(Scanner scan, Showroom showroom)
+ void : updateCarPriceByManager(Scanner scan, Showroom showroom)
+ double : getValidatedDoubleInput(Scanner scan, String prompt, double min, double max)
+ int : getValidatedIntInput(Scanner scan, String prompt, int min, int max)
+ String : getValidatedStringInput(Scanner scan, String prompt)

Data Structures

ArrayList

```
package datastructures;


public class ArrayList<T> {
   private static final int DEFAULT_CAPACITY = 10;
   private Object[] elements;
   private int size;


   public ArrayList() {this(DEFAULT_CAPACITY);}
   public ArrayList(int initialCapacity) {
      if (initialCapacity <= 0) {throw new IllegalArgumentException("Capacity must be greater than 0.");}
      elements = new Object[initialCapacity];
      size = 0;
   }


   public void add(T element) {
      if (size == elements.length) {resizeArray(); }
      elements[size++] = element;
   }


   public void add(int index, T element) {
      if (index < 0 || index > size) {throw new IndexOutOfBoundsException("Index: " + index + ", Size: " +
size);}
      if (size == elements.length) {resizeArray();}
      for (int i = size; i > index; i--) {elements[i] = elements[i - 1];}
      elements[index] = element;
      size++;
   }


   @SuppressWarnings("unchecked")
   public T get(int index) {
```

```
    if (index < 0 || index >= size) {throw new IndexOutOfBoundsException("Index: " + index + ", Size: " +
size);}
    return (T) elements[index];
  }

  @SuppressWarnings("unchecked")
  public T remove(int index) {
    if (index < 0 || index >= size) {throw new IndexOutOfBoundsException("Index: " + index + ", Size: " +
size);}
    T removedElement = (T) elements[index];
    for (int i = index; i < size - 1; i++) {elements[i] = elements[i + 1];}
    elements[--size] = null;
    return removedElement;
  }

  public boolean contains(T element) {return indexOf(element) >= 0;}
  public int indexOf(T element) {
    for (int i = 0; i < size; i++) {
      if (elements[i].equals(element)) {return i;}
    }
    return -1;
  }

  public int size() {return size;}
  public boolean isEmpty() {return size == 0;}

  public void clear() {
    for (int i = 0; i < size; i++) {elements[i] = null;}
    size = 0;
  }

  public void display() {
    System.out.print("ArrayList: [");
    for (int i = 0; i < size; i++) {
```

```
            System.out.print(elements[i]);
            if (i < size - 1) {System.out.print(", ");}
        }
        System.out.println("]");
    }

    private void resizeArray() {
        int newCapacity = elements.length * 2;
        Object[] newArray = new Object[newCapacity];
        for (int i = 0; i < size; i++) {newArray[i] = elements[i];}
        elements = newArray;
    }

    public Object[] toArray() {
        Object[] newArray = new Object[size];
        for (int i = 0; i < size; i++) {newArray[i] = elements[i];}
        return newArray;
    }

    public void set(int index, T element) {
        if (index < 0 || index >= size) {throw new IndexOutOfBoundsException("Index: " + index + ", Size: " +
size);}
        elements[index] = element;
    }

}
```

ArrayQueue

```
package datastructures;

public class ArrayQueue<T> {
    private static final int DEFAULT_CAPACITY = 10;
    private Object[] array;
```

```
private int front;
private int rear;
private int size;

public ArrayQueue() {this(DEFAULT_CAPACITY);}
public ArrayQueue(int capacity) {
  if (capacity <= 0) {throw new IllegalArgumentException("Capacity must be greater than 0.");}
  array = new Object[capacity];
  front = 0;
  rear = -1;
  size = 0;
}

public void enqueue(T item) {
  if (item == null) {throw new IllegalArgumentException("Item cannot be null.");}
  if (isFull()) {resizeArray();}
  rear = (rear + 1) % array.length;
  array[rear] = item;
  size++;
}

@SuppressWarnings("unchecked")
public T dequeue() {
  if (isEmpty()) {throw new IllegalStateException("Queue is empty");}
  T item = (T) array[front];
  array[front] = null;
  front = (front + 1) % array.length;
  size--;
  return item;
}

@SuppressWarnings("unchecked")
public T peek() {
```

```
        if (isEmpty()) {throw new IllegalStateException("Queue is empty");}
        return (T) array[front];
    }

    public boolean isEmpty() {return size == 0;}
    private boolean isFull() {return size == array.length;}
    public int size() {return size;}

    private void resizeArray() {
        int newCapacity = array.length * 2;
        Object[] newArray = new Object[newCapacity];
        int index = 0;

        for (int i = front; index < size; i++) {newArray[index++] = array[i % array.length];}

        array = newArray;
        front = 0;
        rear = size - 1;
    }

    public void displayQueue() {
        if (isEmpty()) {
            System.out.println("The queue is empty.");
            return;
        }
        System.out.println("Queue elements:");
        for (int i = 0; i < size; i++) {
            System.out.print(array[(front + i) % array.length] + " ");
        }
        System.out.println();
    }
}
```

AVLTree

```
package datastructures;


public class AVLTree<T extends Comparable<T>> {
  private class Node {
    T data;
    Node left, right;
    int height;

    public Node(T data) {
      this.data = data;
      this.height = 1;
    }
  }

  private Node root;
  private int height(Node node) {return (node == null) ? 0 : node.height;}
  private int getBalance(Node node) {return (node == null) ? 0 : height(node.left) - height(node.right);}

  private Node rotateRight(Node y) {
    Node x = y.left;
    Node T2 = x.right;
    x.right = y;
    y.left = T2;
    y.height = Math.max(height(y.left), height(y.right)) + 1;
    x.height = Math.max(height(x.left), height(x.right)) + 1;
    return x;
  }

  private Node rotateLeft(Node x) {
    Node y = x.right;
    Node T2 = y.left;
    y.left = x;
```

```
    x.right = T2;
    x.height = Math.max(height(x.left), height(x.right)) + 1;
    y.height = Math.max(height(y.left), height(y.right)) + 1;
    return y;
}

public void insert(T data) {root = insertRec(root, data);}
private Node insertRec(Node node, T data) {
    if (node == null) {return new Node(data);}

    if (data.compareTo(node.data) < 0) {node.left = insertRec(node.left, data);}
    else if (data.compareTo(node.data) > 0) {node.right = insertRec(node.right, data);}
    else {return node;}

    node.height = 1 + Math.max(height(node.left), height(node.right));
    return balance(node);
}

private Node balance(Node node) {
    int balance = getBalance(node);

    if (balance > 1) {
        if (getBalance(node.left) < 0) {node.left = rotateLeft(node.left);}
        return rotateRight(node);
    }
    if (balance < -1) {
        if (getBalance(node.right) > 0) {node.right = rotateRight(node.right);}
        return rotateLeft(node);
    }
    return node;
}

public boolean search(T data) {return searchRec(root, data);}
```

```
private boolean searchRec(Node node, T data) {
   if (node == null) return false;
   if (data.compareTo(node.data) < 0) return searchRec(node.left, data);
   else if (data.compareTo(node.data) > 0) return searchRec(node.right, data);
   else return true;
}

private Node findMin(Node node) {
   while (node.left != null) {node = node.left;}
   return node;
}

public void delete(T data) {root = deleteRec(root, data);}
private Node deleteRec(Node node, T data) {
   if (node == null) return node;

   if (data.compareTo(node.data) < 0) {node.left = deleteRec(node.left, data);}
   else if (data.compareTo(node.data) > 0) {node.right = deleteRec(node.right, data);}
   else {
      if (node.left == null) {return node.right;}
      else if (node.right == null) {return node.left;}

      Node temp = findMin(node.right);
      node.data = temp.data;
      node.right = deleteRec(node.right, temp.data);
   }
   node.height = 1 + Math.max(height(node.left), height(node.right));
   return balance(node);
}

public void inOrderTraversal() {inOrderRec(root);}
private void inOrderRec(Node node) {
   if (node != null) {
```

```
        inOrderRec(node.left);
        System.out.println(node.data);
        inOrderRec(node.right);
      }
    }
}
```

DoublyLinkedList

```
package datastructures;

import java.util.Iterator;
import java.util.NoSuchElementException;

public class DoublyLinkedList<T> implements Iterable<T> {

  private class Node {
    T data;
    Node next;
    Node prev;

    Node(T data) {
      this.data = data;
      this.next = null;
      this.prev = null;
    }
  }

  private Node head;
  private Node tail;
  private int size;

  public DoublyLinkedList() {
    this.head = null;
```

```
    this.tail = null;
    this.size = 0;
}

public void add(T data) {
    Node newNode = new Node(data);
    if (head == null) {
        head = tail = newNode;
    } else {
        tail.next = newNode;
        newNode.prev = tail;
        tail = newNode;
    }
    size++;
}

public void clear() {
    head = null;
    tail = null;
    size = 0;
}

public boolean remove(T data) {
    Node current = head;
    while (current != null) {
        if (current.data.equals(data)) {
            removeNode(current);
            return true;
        }
        current = current.next;
    }
    return false;
}
```

```
private void removeNode(Node node) {
    if (node == head) {
        head = node.next;
        if (head != null) head.prev = null;
    } else if (node == tail) {
        tail = node.prev;
        if (tail != null) tail.next = null;
    } else {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }
    size--;
}

public void display() {
    Node current = head;
    while (current != null) {
        System.out.println(current.data);
        current = current.next;
    }
}

public int size() {return size;}
public T getFirst() {
    if (head == null) throw new NoSuchElementException("List is empty.");
    return head.data;
}

public T getLast() {
    if (tail == null) throw new NoSuchElementException("List is empty.");
    return tail.data;
}
```

```
public T searchById(int id) {
   Node current = head;
   while (current != null) {
      if (current.data.hashCode() == id) {return current.data;}
      current = current.next;
   }
   return null;
}

public boolean isEmpty() {return size == 0;}
public ListIterator getIterator() {return new ListIterator();}

@Override
public Iterator<T> iterator() {return new ListIterator();}

public class ListIterator implements Iterator<T> {
   private Node current;
   private Node lastReturned;

   public ListIterator() {
      this.current = head;
      this.lastReturned = null;
   }

   @Override
   public boolean hasNext() {return current != null;}

   @Override
   public T next() {
      if (!hasNext()) {throw new NoSuchElementException("No more elements in the list.");}
      lastReturned = current;
      T data = current.data;
```

```
        current = current.next;
        return data;
    }

    public boolean hasPrevious() {return lastReturned != null && lastReturned.prev != null;}
    public T previous() {
        if (!hasPrevious()) {throw new NoSuchElementException("No previous elements in the list.");}
        current = lastReturned.prev;
        lastReturned = current;
        return current.data;
    }

    @Override
    public void remove() {
        if (lastReturned == null) {throw new IllegalStateException("No element to remove.");}
        Node nextNode = lastReturned.next;
        removeNode(lastReturned);

        if (current == lastReturned) {current = nextNode;}
        lastReturned = null;
    }
  }
}
```

HashMap

```
package datastructures;

public class HashMap<K, V> {
  private static final int DEFAULT_CAPACITY = 16;
  private static final double LOAD_FACTOR = 0.75;

  private ArrayList<Entry<K, V>> table;
  private int size;
```

```
static class Entry<K, V> {
   K key;
   V value;
   Entry<K, V> next;

   public Entry(K key, V value) {
      this.key = key;
      this.value = value;
      this.next = null;
   }
}

public HashMap() {
   table = new ArrayList<>(DEFAULT_CAPACITY);
   for (int i = 0; i < DEFAULT_CAPACITY; i++) {table.add(null); }
   size = 0;
}

private int hash(K key) {return Math.abs(key.hashCode()) % table.size();}
public void put(K key, V value) {
   int index = hash(key);
   Entry<K, V> newEntry = new Entry<>(key, value);

   if (table.get(index) == null) {table.set(index, newEntry);}
   else {
      Entry<K, V> current = table.get(index);
      while (current.next != null) {
         if (current.key.equals(key)) {
            current.value = value;
            return;
         }
         current = current.next;
      }
```

```
            current.next = newEntry;
        }
        size++;
        if ((double) size / table.size() > LOAD_FACTOR) {resize();}
    }

    public V get(K key) {
        int index = hash(key);
        Entry<K, V> current = table.get(index);

        while (current != null) {
            if (current.key.equals(key)) {return current.value;}
            current = current.next;
        }
        return null;
    }

    public void remove(K key) {
        int index = hash(key);
        Entry<K, V> current = table.get(index);
        Entry<K, V> prev = null;

        while (current != null) {
            if (current.key.equals(key)) {
                if (prev == null) {table.set(index, current.next);}
                else {prev.next = current.next;}
                size--;
                return;
            }
            prev = current;
            current = current.next;
        }
    }
```

```
    public boolean containsKey(K key) {return get(key) != null;}
    private void resize() {
        ArrayList<Entry<K, V>> oldTable = table;
        table = new ArrayList<>(oldTable.size() * 2);
        size = 0;

        for (int i = 0; i < oldTable.size(); i++) {table.add(null);}
        for (int i = 0; i < oldTable.size(); i++) {
            Entry<K, V> entry = oldTable.get(i);
            while (entry != null) {
                put(entry.key, entry.value);
                entry = entry.next;
            }
        }
    }
    public int size() {return size;}
}
```

Now Class of the Project

CAR CLASS

```
public class Car implements Comparable<Car> {
    private int carId;
    private String make;
    private String model;
    private double price;
    private Engine engine;

    public Car(int carId, String make, String model, double price, String engineType, int horsepower) {
        setCarId(carId);
        setMake(make);
        setModel(model);
        setPrice(price);
```

```
    this.engine = new Engine(engineType, horsepower);
  }

  public void setCarId(int carId) {
    if (carId <= 0) {throw new IllegalArgumentException("Car ID must be positive.");}
    this.carId = carId;
  }

  public void setMake(String make) {
    if (make == null || make.trim().isEmpty()) {throw new IllegalArgumentException("Make cannot be empty.");}
    this.make = make;
  }

  public void setModel(String model) {
    if (model == null || model.trim().isEmpty()) {throw new IllegalArgumentException("Model cannot be empty.");}
    this.model = model;
  }

  public void setPrice(double price) {
    if (price < 0) {throw new IllegalArgumentException("Price cannot be negative.");}
    this.price = price;
  }

  // Getters
  public int getCarId() { return this.carId; }
  public String getMake() { return this.make; }
  public String getModel() { return this.model; }
  public double getPrice() { return this.price; }
  public Engine getEngine() { return this.engine; }

  @Override
```

```
public int compareTo(Car other) {
   int priceComparison = Double.compare(this.price, other.price);
   if (priceComparison != 0) {return priceComparison;}
   return this.model.compareTo(other.model);
}

public void display() {
   System.out.println("Car ID: " + this.carId +
         "\nMake: " + this.make +
         "\nModel: " + this.model +
         "\nPrice: $" + String.format("%.2f", this.price));
   this.engine.display();
}

@Override
public String toString() {
   return "Car : \n" +
         "ID : " + carId +
         "\nMake : " + make +
         "\nModel : " + model +
         "\nPrice : " + String.format("%.2f", price) +
         "\nEngine : " + engine.toString();
}

@Override
public boolean equals(Object obj) {
   if (this == obj) return true;
   if (obj == null || getClass() != obj.getClass()) return false;
   Car car = (Car) obj;
   return carId == car.carId;
}

@Override
```

```
    public int hashCode() {return Integer.hashCode(carId);}
}
```

CART CLASS

```
import datastructures.DoublyLinkedList;

class Cart {
    private DoublyLinkedList<Car> items;
    public Cart() {this.items = new DoublyLinkedList<>();}

    public void addToCart(Car car) {
        if (car != null) {
            items.add(car);
            System.out.println(car.getMake() + " " + car.getModel() + " added to the cart.");
        } else {
            System.out.println("Cannot add null to the cart.");
        }
    }

    public void removeFromCart(Car car) {
        if (items.remove(car)) {System.out.println(car.getMake() + " " + car.getModel() + " removed from the
cart.");}
        else {System.out.println("Car not found in the cart.");}
    }

    public void clearCart() {
        items.clear();
        System.out.println("Cart has been cleared.");
    }

    public boolean isEmpty() {return items.isEmpty();}
    public DoublyLinkedList<Car> getItems() {return items;}

    public double calculateTotal() {
```

```
        double total = 0.0;
        DoublyLinkedList<Car>.ListIterator iterator = items.getIterator();
        while (iterator.hasNext()) {
            total += iterator.next().getPrice();
        }
        return total;
    }

    public void displayCart() {
        if (items.isEmpty()) {
            System.out.println("Cart is empty.");
        }else{
                System.out.println("-------------------------");
                System.out.println("Items in the Cart:");
                items.display();
                System.out.println("-------------------------");
        }
    }
}
```

CUSTOMER CLASS

```
import datastructures.DoublyLinkedList;

public class Customer extends Person {
    private int customerId;
    private String username;
    private String password;
    private DoublyLinkedList<Car> ownedCars;
    private DoublyLinkedList<Invoice> purchaseHistory;

    public Customer() {
        super();
        initialize();
```

```
    setCustomerId(0);
    setUsername("");
    setPassword("");
  }

  public Customer(String name, long phoneNumber, int customerId, String username, String password) {
    super(name, phoneNumber);
    initialize();
    setCustomerId(customerId);
    setUsername(username);
    setPassword(password);
  }

  private void initialize() {
    this.ownedCars = new DoublyLinkedList<>();
    this.purchaseHistory = new DoublyLinkedList<>();
  }

  public void setCustomerId(int customerId) {
    if (customerId <= 0) {throw new IllegalArgumentException("Customer ID must be positive.");}
    this.customerId = customerId;
  }

  public void setUsername(String username) {
    if (username == null || username.trim().isEmpty()) {throw new IllegalArgumentException("Username cannot be null or empty.");}
    this.username = username;
  }

  public void setPassword(String password) {
    if (password == null || password.length() < 8) {throw new IllegalArgumentException("Password must exceed 8 characters.");}
    boolean hasUpperCase = false, hasLowerCase = false, hasDigit = false, hasSpecialChar = false;
```

```
    for (char c : password.toCharArray()) {
      if (Character.isUpperCase(c)) hasUpperCase = true;
      else if (Character.isLowerCase(c)) hasLowerCase = true;
      else if (Character.isDigit(c)) hasDigit = true;
      else if (!Character.isLetterOrDigit(c)) hasSpecialChar = true;
    }

    if (!(hasUpperCase && hasLowerCase && hasDigit && hasSpecialChar)) {
      throw new IllegalArgumentException(
        "Password must contain at least one uppercase letter, one lowercase letter, one digit, and one special character."
      );
    }

    this.password = password;
  }

  public int getCustomerId() {return customerId;}
  public String getUsername() {return username;}
  public String getPassword() {return password;}
  public DoublyLinkedList<Car> getOwnedCars() {return ownedCars;}

  public void addPurchaseHistory(Invoice invoice) {
    if (invoice == null) throw new IllegalArgumentException("Invoice cannot be null.");
    this.purchaseHistory.add(invoice);  // ✅ Now it will not be null
  }

  public DoublyLinkedList<Invoice> getPurchaseHistory() { return purchaseHistory; }

  public void addCar(Car car) {
    if (car == null) {throw new IllegalArgumentException("Car cannot be null.");}
    if (ownedCars.searchById(car.getCarId()) != null) {
      throw new IllegalArgumentException("Car with ID " + car.getCarId() + " is already owned by the customer.");
```

```
        }
        ownedCars.add(car);
    }

    @Override
    public void display() {
        System.out.println("\nCustomer Details:");
        super.display();
        System.out.println("Customer ID: " + this.getCustomerId() +
                           "\nUsername: " + this.getUsername() +
                           "\nPassword: " + this.getPassword() +
                           "\nOwned Cars:");

        if (ownedCars.isEmpty()) {System.out.println("No cars owned.");}
        else {ownedCars.display();}
    }
}
```

EMPLOYEE CLASS

```
public class Employee extends Person {
    private int empId;
    private String username;
    private String password;
    private double salary;

    public Employee(String name, long phone, int empId, String username, String password, double salary)
{
        super(name, phone);
        setEmpId(empId);
        setUsername(username);
        setPassword(password);
        setSalary(salary);
```

```
  }

  public void setEmpId(int empId) {
    if (empId <= 0) {throw new IllegalArgumentException("Employee ID must be positive.");}
    this.empId = empId;
  }

  public void setUsername(String username) {
    if (username == null || username.trim().isEmpty()) {
      throw new IllegalArgumentException("Username cannot be null or empty.");
    }
    this.username = username;
  }

  public void setPassword(String password) {
    if (password == null || password.length() < 8) {
      throw new IllegalArgumentException("Password must exceed 8 characters.");
    }

    boolean hasUpperCase = false, hasLowerCase = false, hasDigit = false, hasSpecialChar = false;

    for (char c : password.toCharArray()) {
      if (Character.isUpperCase(c)) hasUpperCase = true;
      else if (Character.isLowerCase(c)) hasLowerCase = true;
      else if (Character.isDigit(c)) hasDigit = true;
      else if (!Character.isLetterOrDigit(c)) hasSpecialChar = true;
    }

    if (!(hasUpperCase && hasLowerCase && hasDigit && hasSpecialChar)) {
      throw new IllegalArgumentException(
        "Password must contain at least one uppercase letter, one lowercase letter, one digit, and one special character."
      );
```

```
    }
    this.password = password;
  }

  public void setSalary(double salary) {
    if (salary < 0) {throw new IllegalArgumentException("Salary cannot be negative.");}
    this.salary = salary;
  }

  public int getEmpId() {return this.empId;}
  public String getUsername() {return this.username;}
  public String getPassword() {return this.password;}
  public double getSalary() {return this.salary;}

  @Override
  public void display() {
    System.out.println("Employee Details:");
    super.display();
    System.out.println("Emp ID: " + this.empId +
                "\nUsername: " + this.username +
                "\nSalary: " + this.salary);
  }

  @Override
  public String toString() {
    return "Employee {" +
        "Emp ID=" + empId +
        ", Name=" + getName() +
        ", Phone=" + getPhone() +
        ", Username='" + username + '\'' +
        ", Salary=" + salary +
       '}';
}}
```

ENGINE CLASS

```
public class Engine {
   private String type;
   private int horsepower;

   public Engine(String type, int horsepower) {
      setType(type);
      setHorsepower(horsepower);
   }

   public void setType(String type) {
      if (type == null || type.trim().isEmpty()) {
         throw new IllegalArgumentException("Engine type cannot be empty.");
      }
      this.type = type;
   }

   public void setHorsepower(int horsepower) {
      if (horsepower <= 0) {
         throw new IllegalArgumentException("Horsepower must be positive.");
      }
      this.horsepower = horsepower;
   }

   public void display() {System.out.println("Engine Type: " + this.type + "\nHorsepower: " +
this.horsepower);}
   @Override
   public String toString() {
      return "Engine Type: " + this.type + "\nHorsepower: " + this.horsepower;
   }
}
```

INVOICE CLASS

```
import datastructures.DoublyLinkedList;


class Invoice {
   private String customerName;
   private DoublyLinkedList<Car> purchasedCars;
   private double totalAmount;
   private Payment paymentDetails;


   public Invoice(String customerName, DoublyLinkedList<Car> purchasedCars, double totalAmount, Payment
paymentDetails) {
      this.customerName = customerName;
      this.purchasedCars = new DoublyLinkedList<>();


      DoublyLinkedList<Car>.ListIterator iterator = purchasedCars.getIterator();
      while (iterator.hasNext()) {
         this.purchasedCars.add(iterator.next());
      }


      this.totalAmount = totalAmount;
      this.paymentDetails = paymentDetails;
   }


   public String getCustomerName() {return customerName;}
   public DoublyLinkedList<Car> getPurchasedCars() {return purchasedCars;}
   public double getTotalAmount() {return totalAmount;}
   public Payment getPaymentDetails() {return paymentDetails;}


   @Override
   public String toString() {
      StringBuilder invoice = new StringBuilder();
      invoice.append("Invoice for: ").append(customerName).append("\n")
            .append("Purchased Cars:\n");


      DoublyLinkedList<Car>.ListIterator iterator = purchasedCars.getIterator();
```

```
    while (iterator.hasNext()) {
      Car car = iterator.next();
      invoice.append("  - ").append(car.getMake())
          .append(" ").append(car.getModel())
          .append(", Price: ").append(car.getPrice()).append("\n");
    }

    invoice.append("Total Amount: ").append(totalAmount).append("\n")
        .append("Payment Details:\n")
        .append(paymentDetails.toString()).append("\n");
    return invoice.toString();
  }
}
```

MANAGER CLASS

```
public class Manager extends Employee {
  private String department;

  public Manager(String name, long phone, int empId, String username, String password, double salary, String
department) {
    super(name, phone, empId, username, password, salary);
    setDepartment(department);
  }

  public void setDepartment(String department) {
    if (department == null || department.trim().isEmpty()) {
      throw new IllegalArgumentException("Department cannot be null or empty.");
    }
    this.department = department;
  }

  public String getDepartment() {return this.department;}
  @Override
  public void setPassword(String password) {super.setPassword(password);}
```

```
    @Override
    public void display() {
        super.display();
        System.out.println("Department: " + this.department);
    }

    @Override
    public String toString() {
        return "Manager {" +
                "Emp ID=" + getEmpId() +
                ", Name=" + getName() +
                ", Phone=" + getPhone() +
                ", Username='" + getUsername() + '\'' +
                ", Salary=" + getSalary() +
                ", Department='" + this.department + '\'' +
                '}';
    }
}
```

PAYMENT CLASS

```
public class Payment {
    private String cardNumber;
    private String cardHolderName;
    private double amount;

    public Payment(String cardNumber, String cardHolderName, double amount) {
        setCardNumber(cardNumber);
        setCardHolderName(cardHolderName);
        setAmount(amount);
    }

    public void setCardNumber(String cardNumber) {
```

```
    if (cardNumber == null) {throw new IllegalArgumentException("Invalid card number. It must be a 16-digit number.");}
    String sanitizedCardNumber = cardNumber.replaceAll("\\s", "");
    if (!sanitizedCardNumber.matches("\\d{16}")) {
      throw new IllegalArgumentException("Invalid card number. It must be a 16-digit number.");
    }
    this.cardNumber = sanitizedCardNumber;
  }

  public void setCardHolderName(String cardHolderName) {
    if (cardHolderName == null || cardHolderName.trim().isEmpty()) {
      throw new IllegalArgumentException("Cardholder name cannot be null or empty.");
    }
    this.cardHolderName = cardHolderName.trim();
  }

  public void setAmount(double amount) {
    if (amount <= 0) {
      throw new IllegalArgumentException("Amount must be greater than zero.");
    }
    this.amount = amount;
  }

  public String getCardNumber() {return cardNumber;}
  public String getCardHolderName() {return cardHolderName;}
  public double getAmount() {return amount;}

  private String maskCardNumber(String cardNumber) {return "**** **** **** " + cardNumber.substring(cardNumber.length() - 4);}

  @Override
  public String toString() {
    return "Payment Details:\n" +
        "Card Holder: " + cardHolderName + "\n" +
```

```
            "Card Number: " + maskCardNumber(cardNumber) + "\n" +
            "Amount Paid: $" + String.format("%.2f", amount);
    }
}
```

PERSON CLASS

```
public class Person {
    private String name;
    private long phone;

    public Person() {
        this.name = "";
        this.phone = 0;
    }

    public Person(String name, long phone) {
        setName(name);
        setPhone(phone);
    }

    public void setName(String name) {
        if (name == null || name.trim().isEmpty()) {
            throw new IllegalArgumentException("Name cannot be empty.");
        }
        this.name = name;
    }

    public void setPhone(long phone) {
        String phoneString = String.valueOf(phone);
        if (phoneString.length() < 10) {
            throw new IllegalArgumentException("Phone number must have at least 10 digits.");
        }
        if (phoneString.startsWith("03")) {
            phoneString = "92" + phoneString.substring(1);
```

```
        }
        if (phoneString.startsWith("92") && phoneString.length() == 12) {
        } else if (phoneString.length() > 12) {
            throw new IllegalArgumentException("Phone number exceeds valid length. Use the format
923XXXXXXXXX.");
        } else {
            throw new IllegalArgumentException("Invalid phone number. Use the format 923XXXXXXXXX.");
        }

        this.phone = Long.parseLong(phoneString);
    }

    public String getName() {return name;}
    public long getPhone() {return phone;}

    public void display() {
        System.out.println("Name: " + this.name);
        System.out.println("Phone: " + this.phone);
    }
}
```

SHOWROOM CLASS

```
    import java.util.Iterator;
    import datastructures.DoublyLinkedList;
    import datastructures.HashMap;
    import datastructures.AVLTree;
    import datastructures.ArrayList;
    import datastructures.ArrayQueue;

    public class Showroom implements Iterable<Employee> {
        private String name;
        private String location;
        private HashMap<Integer, Car> cars;
```

```
    private DoublyLinkedList<Employee> employees;
    private AVLTree<Car> sortedCars;
    private ArrayList<Invoice> invoices;

    public Showroom(String name, String location) {
        this.name = name;
        this.location = location;
        this.cars = new HashMap<>();
        this.employees = new DoublyLinkedList<>();
        this.sortedCars = new AVLTree<>();
        this.invoices = new ArrayList<>();
    }
    public void setInvoices(ArrayQueue<Invoice> invoicesQueue) { while
(!invoicesQueue.isEmpty()) { invoices.add(invoicesQueue.dequeue());}}

    public void addCar(Car car) {
        if (car == null) throw new IllegalArgumentException("Car cannot be null.");
        cars.put(car.getCarId(), car);
        sortedCars.insert(car);
    }

    public boolean removeCar(int carId) {
        Car car = cars.get(carId);
        if (car != null) {
            cars.remove(carId);
            sortedCars.delete(car);
            return true;
        }
        System.out.println("Car not found!");
        return false;
    }
```

```
public boolean updateCarPrice(int carId, double newPrice) {
    Car car = cars.get(carId);
    if (car != null) {
        sortedCars.delete(car);
        car.setPrice(newPrice);
        sortedCars.insert(car);
        System.out.println("Car price updated: " + car.getModel() + " -> " + newPrice);
        return true;
    }
    System.out.println("Car not found!");
    return false;
}

public void addEmployee(Employee employee) {
    if (employee == null) throw new IllegalArgumentException("Employee cannot be null.");
    employees.add(employee);
}
public Car getCarById(int carId) {return cars.get(carId);}

public boolean removeEmployee(int empId) {
    Iterator<Employee> iterator = employees.getIterator();
    while (iterator.hasNext()) {
        Employee employee = iterator.next();
        if (employee.getEmpId() == empId) {
            iterator.remove();
            return true;
        }
    }
    System.out.println("Employee not found!");
    return false;
```

```
}

public boolean updateEmployeeSalary(int empId, double newSalary) {
   Iterator<Employee> iterator = employees.getIterator();
   while (iterator.hasNext()) {
      Employee employee = iterator.next();
      if (employee.getEmpId() == empId) {
         employee.setSalary(newSalary);
         System.out.println("Salary updated for " + employee.getName() + " -> " + newSalary);
         return true;
      }
   }
   System.out.println("Employee not found!");
   return false;
}

public void addInvoice(Invoice invoice) {
   if (invoice == null) throw new IllegalArgumentException("Invoice cannot be null.");
   invoices.add(invoice);
   System.out.println("Invoice added successfully.");
}

public void printInvoices() {
   if (invoices.isEmpty()) {
      System.out.println("No invoices available.");}
      else {
      System.out.println("Invoices:");
      for (int i = 0; i < invoices.size(); i++) {
         System.out.println("-------------");
         System.out.println(invoices.get(i));
         System.out.println("-------------");
```

```
        }
      }
    }

    public void display() {
      System.out.println("Showroom Name: " + this.name);
      System.out.println("Location: " + this.location);
      System.out.println("\nCars in the showroom:");
      sortedCars.inOrderTraversal();
      System.out.println("\nEmployees in the showroom:");
      printEmployees();
      printInvoices();
    }

    public void printEmployees() {
      if (employees.isEmpty()) {System.out.println("No employees available in the showroom.");}
      else {
        Iterator<Employee> iterator = employees.getIterator();
        while (iterator.hasNext()) {
          Employee employee = iterator.next();
          employee.display();
          System.out.println();
        }
      }
    }

    public void printSortedCars() {
      System.out.println("Cars sorted by price/model:");
      sortedCars.inOrderTraversal();
    }
```

```
    @Override
    public Iterator<Employee> iterator() {return employees.getIterator();}
}
```

USINGSHOWROOM CLASS

```
import datastructures.ArrayQueue;
import datastructures.DoublyLinkedList;
import java.util.InputMismatchException;
import java.util.Scanner;

public class UsingShowroom {
  public static void main(String[] args) {
    Scanner scan = new Scanner(System.in);

    // Predefined data
    DoublyLinkedList<Manager> managers = new DoublyLinkedList<>();
    managers.add(new Manager("Boss", 923165498760L, 51, "admin", "12P@$$w0rd", 75000, "Graphics"));

    DoublyLinkedList<Customer> customers = new DoublyLinkedList<>();
    customers.add(new Customer("Abdullah Asif", 923224332434L, 101, "aby", "P@$$w0rd"));
    customers.add(new Customer("Zohair Shazib", 923492342421L, 102, "zoi@cus", "12P@$$w0rd"));

    DoublyLinkedList<Employee> employees = new DoublyLinkedList<>();
    employees.add(new Employee("Ali", 923556655660L, 1, "ali", "12P@$$w0rd", 50000));
    employees.add(new Employee("Ali", 923556655660L, 1, "ali@emp", "12P@$$w0rd", 50000));
    employees.add(new Employee("Asad", 923228822880L, 2, "asad@emp", "12P@$$w0rd", 60000));
    employees.add(new Employee("Hassan", 923312345678L, 3, "hassan@emp", "12P@$$w0rd", 55000));
    employees.add(new Employee("Sara", 923301234567L, 4, "sara@emp", "12P@$$w0rd", 45000));
    employees.add(new Employee("Ayesha", 923332112233L, 5, "ayesha@emp", "12P@$$w0rd", 48000));
    employees.add(new Employee("Kamran", 923345678901L, 6, "kamran@emp", "12P@$$w0rd", 52000));

    Showroom showroom = new Showroom("Nova Showroom", "GRW");
```

```
showroom.addCar(new Car(1, "Toyota", "Camry", 24000, "V6", 3000));
showroom.addCar(new Car(2, "Toyota", "Camry", 24000, "V6", 3000));
showroom.addCar(new Car(3, "Tesla", "Model 3", 35000, "Electric", 2000));
showroom.addCar(new Car(4, "Honda", "Civic", 20000, "V4", 1500));
showroom.addCar(new Car(5, "BMW", "X5", 60000, "V8", 4000));
showroom.addCar(new Car(6, "Mercedes", "E-Class", 55000, "V6", 3500));

ArrayQueue<Invoice> invoices = new ArrayQueue<>();
DoublyLinkedList<Car> invoice1Cars = new DoublyLinkedList<>();

invoice1Cars.add(new Car(1, "Toyota", "Camry", 24000, "V6", 300));

Invoice invoice1 = new Invoice(
   "Abdullah Asif",
   invoice1Cars,
   24000,
   new Payment("1234567891011121", "Abdullah Asif", 24000)
);

DoublyLinkedList<Car> invoice2Cars = new DoublyLinkedList<>();
invoice2Cars.add(new Car(2, "Tesla", "Model 3", 35000, "Electric", 200));

Invoice invoice2 = new Invoice(
   "Zohair Shazib",
   invoice2Cars,
   35000,
   new Payment("5678123491011121", "Zohair Shazib", 35000)
);

invoices.enqueue(invoice1);
invoices.enqueue(invoice2);
showroom.setInvoices(invoices);

System.out.println("Welcome to the Nova Showroom Management System!");
```

```
    boolean exit = false;
    while (!exit) {
      System.out.println("\nChoose an option:");
      System.out.println("1. Sign Up (Customers Only)");
      System.out.println("2. Log In");
      System.out.println("3. Exit");

      int mainOption = getValidatedIntInput(scan, "Enter your choice: ", 1, 3);
      scan.nextLine();

      switch (mainOption) {
        case 1:
          signUpCustomer(scan, customers);
          break;
        case 2:
          logInUser(scan, managers, employees, customers, showroom);
          break;
        case 3:
          System.out.println("Thank you for using the Nova Showroom Management System. Goodbye!");
          exit = true;
          break;
        default:
          System.out.println("Invalid option. Please try again.");
      }
    }
    scan.close();
  }

  private static void logInUser(Scanner scan, DoublyLinkedList<Manager> managers, DoublyLinkedList<Employee>
employees, DoublyLinkedList<Customer> customers, Showroom showroom) {
    System.out.println("Choose account type to log in:");
    System.out.println("1. Manager\n2. Employee\n3. Customer");
    int loginType = getValidatedIntInput(scan, "Enter your choice: ", 1, 3);
    scan.nextLine();
```

```
    String loginUsername = getValidatedStringInput(scan, "Enter your username: ").trim();
    String loginPassword = getValidatedStringInput(scan, "Enter your password: ").trim();

    boolean loggedIn = false;
    if (loginType == 1) {loggedIn = managerLogin(scan, managers, showroom, loginUsername, loginPassword);}
    else if (loginType == 2) {loggedIn = employeeLogin(scan, employees, showroom, loginUsername, loginPassword);}
    else if (loginType == 3) {loggedIn = customerLogin(scan, customers, showroom, loginUsername, loginPassword);}
    if (!loggedIn) {System.out.println("Invalid username or password. Please try again.");}
  }

    private static boolean managerLogin(Scanner scan, DoublyLinkedList<Manager> managers, Showroom showroom, String username, String password) {
    for (Manager manager : managers) {
      if (manager.getUsername().equals(username)) {
        while (!manager.getPassword().equals(password)) {
          System.out.print("Incorrect password. Please try again: ");
          password = scan.nextLine();
        }
        System.out.println("Login successful! Welcome, Manager " + manager.getName());
        managerOperations(scan, showroom, manager);
        return true;
      }
    }
    System.out.println("Username not found!");
    return false;
  }

  private static boolean employeeLogin(Scanner scan, DoublyLinkedList<Employee> employees, Showroom showroom, String username, String password) {
    for (Employee employee : employees) {
      if (employee.getUsername().equals(username)) {
        while (!employee.getPassword().equals(password)) {
          System.out.print("Incorrect password. Please try again: ");
          password = scan.nextLine();
        }
```

```
            System.out.println("Login successful! Welcome, Employee " + employee.getName());
            employeeOperations(scan, showroom, employee);
            return true;
        }
    }
    System.out.println("Username not found!");
    return false;
}

  private static boolean customerLogin(Scanner scan, DoublyLinkedList<Customer> customers, Showroom
showroom, String username, String password) {
    for (Customer customer : customers) {
      if (customer.getUsername().equals(username)) {
        while (!customer.getPassword().equals(password)) {
          System.out.print("Incorrect password. Please try again: ");
          password = scan.nextLine();
        }
        System.out.println("Login successful! Welcome, " + customer.getName());
        customerOperations(scan, showroom, customer);
        return true;
      }
    }
    System.out.println("Username not found!");
    return false;
}


  private static void customerOperations(Scanner scan, Showroom showroom, Customer customer) {
    boolean keepGoing = true;
    DoublyLinkedList<Car> cart = new DoublyLinkedList<>();

    while (keepGoing) {
      System.out.println("\nCustomer Menu:");
      System.out.println("1. View Cars");
      System.out.println("2. Add Car to Cart");
```

```
System.out.println("3. View Cart");
System.out.println("4. Confirm Payment");
System.out.println("5. Update Profile");
System.out.println("6. View Purchase History");
System.out.println("7. Log Out");
int choice = getValidatedIntInput(scan, "Enter your choice: ", 1, 7);

switch (choice) {
  case 1:
    showroom.printSortedCars();
    break;
  case 2:
    int carIdToAdd = getValidatedIntInput(scan, "Enter Car ID to add to cart: ", 1, Integer.MAX_VALUE);
    Car car = showroom.getCarById(carIdToAdd);
    if (car != null) {
      cart.add(car);
      System.out.println("Car added to cart successfully!");
    } else {
      System.out.println("Car not found!");
    }
    break;
  case 3:
    if (cart.isEmpty()) {
      System.out.println("Your cart is empty!");
    } else {
      System.out.println("Your Cart:");
      for (Car c : cart) {
        c.display();
      }
    }
    break;
  case 4:
    if (cart.isEmpty()) {
      System.out.println("Your cart is empty! Add cars to your cart before confirming payment.");
      break;
```

```
    }
    double totalAmount = 0;
    for (Car c : cart) { totalAmount += c.getPrice(); }
    System.out.printf("Total Amount: %.2f\n", totalAmount);

    scan.nextLine();
    System.out.print("Enter Card Holder Name: ");
    String cardHolderName = scan.nextLine();
    System.out.print("Enter Card Number: ");
    String cardNumber = scan.nextLine();

    Payment payment = new Payment(cardNumber, cardHolderName, totalAmount);
    System.out.println("\nPayment Successful!");
    System.out.println(payment);

    Invoice invoice = new Invoice(customer.getName(), cart, totalAmount, payment);
    showroom.addInvoice(invoice);
    customer.addPurchaseHistory(invoice);

    for (Car c : cart) { showroom.removeCar(c.getCarId()); }
    cart.clear();
    break;
  case 5:
    updateCustomerProfile(scan, customer);
    break;
  case 6:
    if (customer.getPurchaseHistory().isEmpty()) {
      System.out.println("No purchase history available.");
      break;
    }
    viewPurchaseHistory(customer);
    break;
  case 7:
    keepGoing = false;
    System.out.println("Logging out...");
```

```
            break;
        default:
            System.out.println("Invalid option. Please try again.");
      }
    }
}


private static void updateCustomerProfile(Scanner scan, Customer customer) {
    System.out.println("\nUpdate Profile");
    System.out.print("Enter your current password: ");
    scan.nextLine();
    String enteredPassword = scan.nextLine();

    if (!customer.getPassword().equals(enteredPassword)) {
        System.out.println("Incorrect password! Profile update failed.");
        return;
    }

    System.out.print("Enter new name (or press Enter to keep current name): ");
    String newName = scan.nextLine().trim();
    if (!newName.isEmpty()) {
        customer.setName(newName);
    }

    System.out.print("Enter new username (or press Enter to keep current username): ");
    String newUsername = scan.nextLine().trim();
    if (!newUsername.isEmpty()) {
        customer.setUsername(newUsername);
    }

    System.out.println("Profile updated successfully!");
}

private static void viewPurchaseHistory(Customer customer) {
```

```
    System.out.println("\nPurchase History for " + customer.getName());

    if (customer.getPurchaseHistory().isEmpty()) {
        System.out.println("No purchase history found.");
        return;
    }

    for (Invoice invoice : customer.getPurchaseHistory()) {
        System.out.println(invoice);
    }
}


private static void signUpCustomer(Scanner scan, DoublyLinkedList<Customer> customers) {
    System.out.println("Enter your full name: ");
    String name = scan.nextLine();
    System.out.println("Enter your phone number: ");
    long phoneNumber = scan.nextLong();
    scan.nextLine();
    System.out.println("Enter your username: ");
    String username = scan.nextLine();
    System.out.println("Enter your password: ");
    String password = scan.nextLine();

    int customerId = customers.size() + 101;
    Customer newCustomer = new Customer(name, phoneNumber, customerId, username, password);
    customers.add(newCustomer);
    System.out.println("Signup successful.....!");
}

private static void addEmployee(Scanner scan, Showroom showroom) {
    System.out.println("Enter Employee Name: ");
    scan.nextLine();
    String name = scan.nextLine();
    int empId = getValidatedIntInput(scan, "Enter Employee ID: ", 1, Integer.MAX_VALUE);
```

```
    System.out.println("Enter Phone Number: ");
    long phoneNumber = scan.nextLong();
    scan.nextLine();

    System.out.println("Enter Username: ");
    String username = scan.nextLine();

    System.out.println("Enter Password: ");
    String password = scan.nextLine();

    System.out.println("Enter Salary: ");
    double salary = scan.nextDouble();
    scan.nextLine();

    Employee employee = new Employee(name, phoneNumber, empId, username, password, salary);
    showroom.addEmployee(employee);
    System.out.println("Employee added successfully!");
}

private static void addCar(Scanner scan, Showroom showroom) {
    int carId = getValidatedIntInput(scan, "Enter Car ID: ", 1, Integer.MAX_VALUE);
    scan.nextLine();

    System.out.println("Enter Make: ");
    String make = scan.nextLine().trim();
    while (make.isEmpty()) {
        System.out.println("Make cannot be empty! Please enter a valid make: ");
        make = scan.nextLine().trim();
    }

    System.out.println("Enter Model: ");
    String model = scan.nextLine().trim();
    while (model.isEmpty()) {
        System.out.println("Model cannot be empty! Please enter a valid model: ");
```

```
        model = scan.nextLine().trim();
    }

    System.out.println("Enter Price: ");
    double price = scan.nextDouble();
    scan.nextLine();

    System.out.println("Enter Engine Type: ");
    String engineType = scan.nextLine().trim();
    while (engineType.isEmpty()) {
        System.out.println("Engine Type cannot be empty! Please enter a valid engine type: ");
        engineType = scan.nextLine().trim();
    }

    int horsepower = getValidatedIntInput(scan, "Enter Horsepower: ", 1, 10000);

    Car car = new Car(carId, make, model, price, engineType, horsepower);
    showroom.addCar(car);
    System.out.println("Car added successfully!");
}

private static void employeeOperations(Scanner scan, Showroom showroom, Employee loggedInEmployee) {
    boolean keepGoing = true;
    while (keepGoing) {
        System.out.println("\nEmployee Menu:");
        System.out.println("1. View Cars");
        System.out.println("2. View Showroom Details");
        System.out.println("3. Sell Car");
        System.out.println("4. Update Profile");
        System.out.println("5. Log Out");

        int choice = getValidatedIntInput(scan, "Enter your choice: ", 1, 5);

        switch (choice) {
            case 1:
```

```
            showroom.printSortedCars();
            break;
        case 2:
            showroom.display();
            break;
        case 3:
            int carIdToSell = getValidatedIntInput(scan, "Enter Car ID to sell: ", 1, Integer.MAX_VALUE);
            Car carToSell = showroom.getCarById(carIdToSell);
            if (carToSell != null) {
                showroom.removeCar(carIdToSell);
                System.out.println("Car sold successfully!");
            }
            else {System.out.println("Car not found!");}
            break;
        case 4:
            updateEmployeeProfile(scan, loggedInEmployee);
            break;
        case 5:
            keepGoing = false;
            System.out.println("Logging out...");
            break;
        default:
            System.out.println("Invalid option. Please try again.");
      }
    }
}

private static void updateEmployeeProfile(Scanner scan, Employee employee) {
    System.out.println("\n--- Update Profile ---");
    scan.nextLine();
    String enteredPassword = getValidatedStringInput(scan, "Enter your password to proceed: ");

    if (!employee.getPassword().equals(enteredPassword)) {
        System.out.println("Incorrect password! Profile update canceled.");
        return;
```

```
    }

    String newName = getValidatedStringInput(scan, "Enter new name (or press Enter to keep current name): ");
    if (!newName.trim().isEmpty()) {
      employee.setName(newName);
      System.out.println("Name updated successfully!");
    }

    String newUsername = getValidatedStringInput(scan, "Enter new username (or press Enter to keep current
username): ");
    if (!newUsername.trim().isEmpty()) {
      employee.setUsername(newUsername);
      System.out.println("Username updated successfully!");
    }

    System.out.println("Profile updated successfully!");
  }

  private static void managerOperations(Scanner scan, Showroom showroom, Manager manager) {
    boolean keepGoing = true;
    while (keepGoing) {
      System.out.println("\nManager Menu:");
      System.out.println("1. Add Employee");
      System.out.println("2. Add Car");
      System.out.println("3. View Showroom Details");
      System.out.println("4. View Invoices");
      System.out.println("5. Remove Employee");
      System.out.println("6. Remove Car");
      System.out.println("7. Update Car Price");
      System.out.println("8. Update Profile");
      System.out.println("9. Log Out");

      int choice = getValidatedIntInput(scan, "Enter your choice: ", 1, 9);

      switch (choice) {
```

```
case 1:
   addEmployee(scan, showroom);
   break;
case 2:
   addCar(scan, showroom);
   break;
case 3:
   showroom.display();
   break;
case 4:
   showroom.printInvoices();
   break;
case 5:
   System.out.println("Enter Employee ID to remove: ");
   int empIdToRemove = scan.nextInt();
   boolean removed = showroom.removeEmployee(empIdToRemove);
   if (removed) {System.out.println("Employee removed successfully!");}
   else {System.out.println("Employee not found!");}
   break;
case 6:
   int carIdToRemove = getValidatedIntInput(scan, "Enter Car ID to remove: ", 1, Integer.MAX_VALUE);
   boolean carRemoved = showroom.removeCar(carIdToRemove);
   if (carRemoved) {System.out.println("Car removed successfully!");}
   else {System.out.println("Car not found!");}
   break;
case 7:
   updateCarPriceByManager(scan, showroom);
   break;
case 8:
   updateManagerProfile(scan, manager);
   break;
case 9:
   keepGoing = false;
   System.out.println("Logging out...");
   break;
```

```
        default:
          System.out.println("Invalid option. Please try again.");
      }
    }
  }

  private static void updateManagerProfile(Scanner scan, Manager manager) {
    System.out.print("\nEnter your current password for verification: ");
    scan.nextLine();
    String password = scan.nextLine();

    if (!manager.getPassword().equals(password)) {
      System.out.println("Incorrect password! Profile update failed.");
      return;
    }

    System.out.println("\nProfile Update Menu:");
    System.out.println("1. Update Name");
    System.out.println("2. Update Username");
    System.out.println("3. Cancel");

    int option = getValidatedIntInput(scan, "Enter your choice: ", 1, 3);

    switch (option) {
      case 1:
        System.out.print("Enter new name: ");
        scan.nextLine();
        String newName = scan.nextLine();
        if (!newName.trim().isEmpty()) {
          manager.setName(newName);
          System.out.println("Name updated successfully!");
        }
        else {System.out.println("Invalid name. Update failed.");}
        break;
      case 2:
```

```
            System.out.print("Enter new username: ");
            String newUsername = scan.nextLine();
            if (!newUsername.trim().isEmpty()) {
                manager.setUsername(newUsername);
                System.out.println("Username updated successfully!");
            }
            else {System.out.println("Invalid username. Update failed.");}
            break;
        case 3:
            System.out.println("Profile update canceled.");
            break;
        default:
            System.out.println("Invalid option.");
    }
}




private static void updateCarPriceByManager(Scanner scan, Showroom showroom) {
    int carId = getValidatedIntInput(scan, "Enter Car ID to update price: ", 1, Integer.MAX_VALUE);
    Car car = showroom.getCarById(carId);

    if (car != null) {
        System.out.println("Current Price: $" + car.getPrice());
        double newPrice = getValidatedDoubleInput(scan, "Enter new price: ", 1, Double.MAX_VALUE);

        boolean updated = showroom.updateCarPrice(carId, newPrice);
        if (updated) {System.out.println("Car price updated successfully!");}
        else {System.out.println("Failed to update car price.");}
    }
    else {System.out.println("Car not found!");}
}

private static double getValidatedDoubleInput(Scanner scan, String prompt, double min, double max) {
```

```
    while (true) {
        try {
            System.out.print(prompt);
            double input = scan.nextDouble();
            if (input < min || input > max) {
                throw new IllegalArgumentException("Input must be between " + min + " and " + max + ".");
            }
            scan.nextLine();
            return input;
        } catch (InputMismatchException e) {
            System.out.println("Invalid input. Please enter a valid number.");
            scan.nextLine();
        }
    }
}



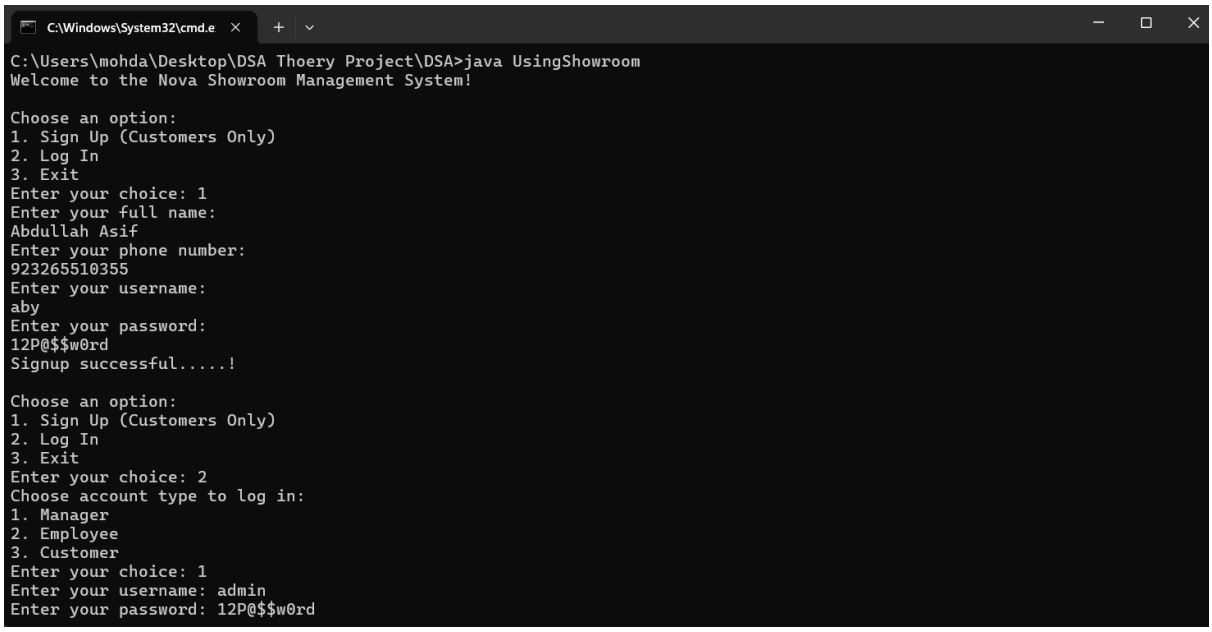
private static int getValidatedIntInput(Scanner scan, String prompt, int min, int max) {
    while (true) {
        try {
            System.out.print(prompt);
            int input = scan.nextInt();
            if (input < min || input > max) {
                throw new IllegalArgumentException("Input must be between " + min + " and " + max + ".");
            }
            return input;
        } catch (InputMismatchException e) {
            System.out.println("Invalid input. Please enter a valid number.");
            scan.nextLine();
        }
    }
}

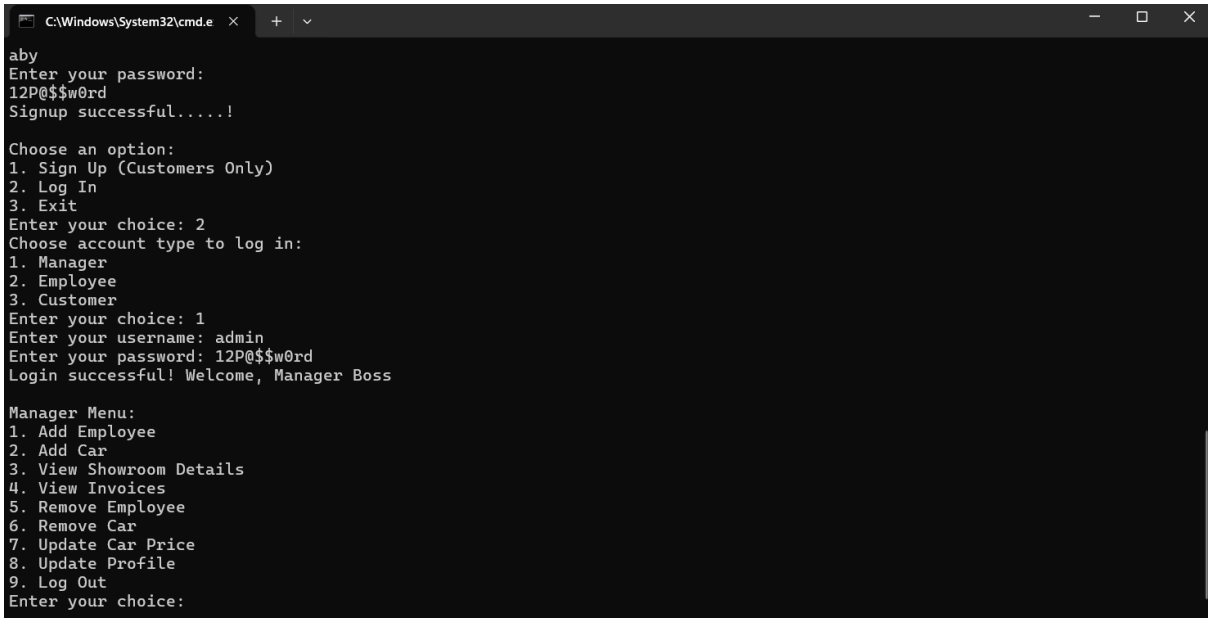
private static String getValidatedStringInput(Scanner scan, String prompt) {
```

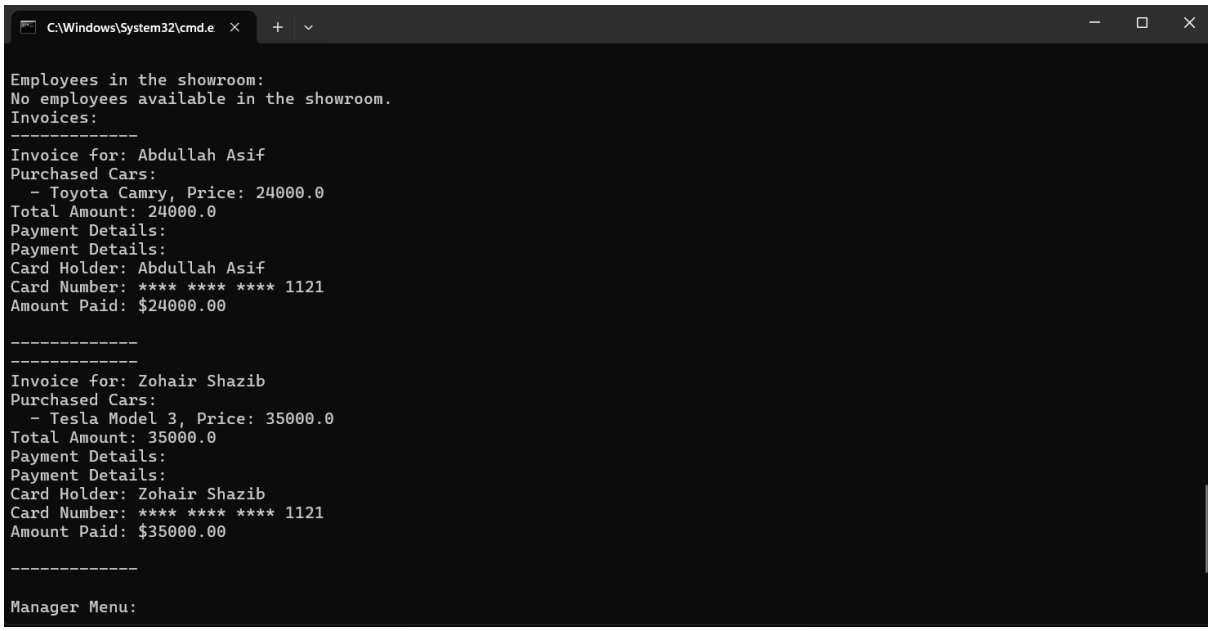
```
        System.out.print(prompt);
        return scan.nextLine().trim();
    }

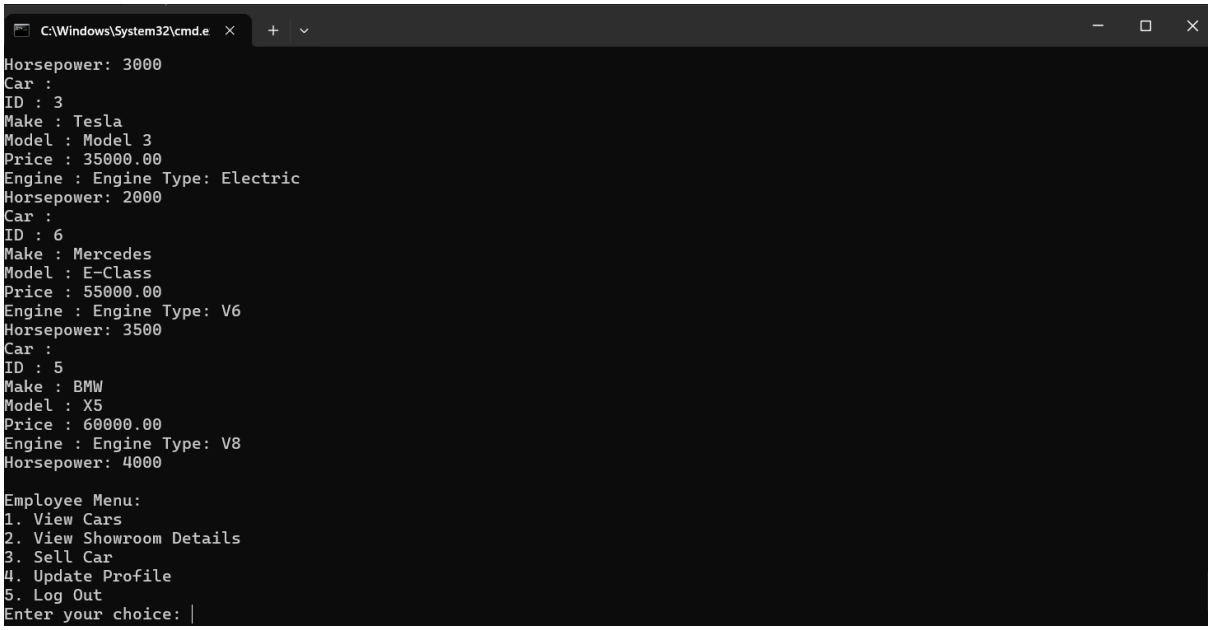

}
```

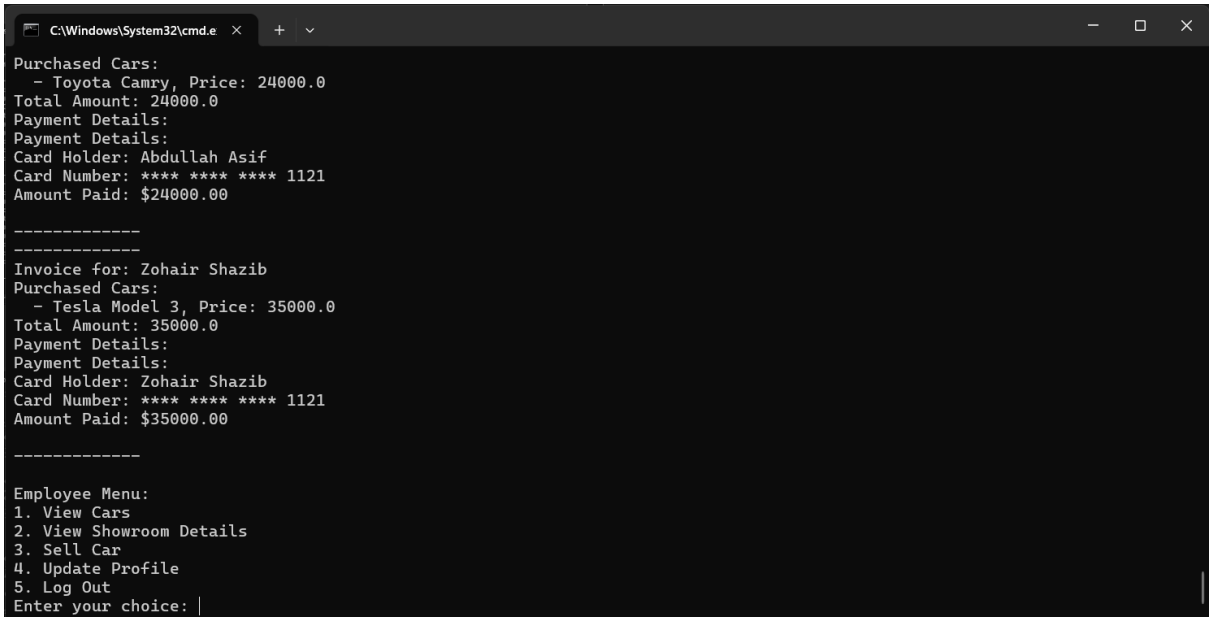
OUTPUT




C:\Windows\System32\cmd.e
aby
Enter your password:
12P@$$w0rd
Signup successful.....!
Choose an option:
1. Sign Up (Customers Only)
2. Log In
3. Exit
Enter your choice: 2
Choose account type to log in:
1. Manager
2. Employee
3. Customer
Enter your choice: 1
Enter your username: admin
Enter your password: 12P@$$w0rd
Login successful! Welcome, Manager Boss
Manager Menu:
1. Add Employee
2. Add Car
3. View Showroom Details
4. View Invoices
5. Remove Employee
6. Remove Car
7. Update Car Price
8. Update Profile
9. Log Out
Enter your choice:


C:\Windows\System32\cmd.e
Employees in the showroom:
No employees available in the showroom.
Invoices:
Invoice for: Abdullah Asif
Purchased Cars:
- Toyota Camry, Price: 24000.0
Total Amount: 24000.0
Payment Details:
Payment Details:
Card Holder: Abdullah Asif
Card Number: **** **** **** 1121
Amount Paid: $24000.00
Invoice for: Zohair Shazib
Purchased Cars:
- Tesla Model 3, Price: 35000.0
Total Amount: 35000.0
Payment Details:
Payment Details:
Card Holder: Zohair Shazib
Card Number: **** **** **** 1121
Amount Paid: $35000.00
Manager Menu:


C:\Windows\System32\cmd.e
Horsepower: 3000
Car :
ID : 3
Make : Tesla
Model : Model 3
Price : 35000.00
Engine : Engine Type: Electric
Horsepower: 2000
Car :
ID : 6
Make : Mercedes
Model : E-Class
Price : 55000.00
Engine : Engine Type: V6
Horsepower: 3500
Car :
ID : 5
Make : BMW
Model : X5
Price : 60000.00
Engine : Engine Type: V8
Horsepower: 4000
Employee Menu:
1. View Cars
2. View Showroom Details
3. Sell Car
4. Update Profile
5. Log Out
Enter your choice:


C:\Windows\System32\cmd.e
Purchased Cars:
- Toyota Camry, Price: 24000.0
Total Amount: 24000.0
Payment Details:
Payment Details:
Card Holder: Abdullah Asif
Card Number: **** **** **** 1121
Amount Paid: $24000.00
Invoice for: Zohair Shazib
Purchased Cars:
- Tesla Model 3, Price: 35000.0
Total Amount: 35000.0
Payment Details:
Payment Details:
Card Holder: Zohair Shazib
Card Number: **** **** **** 1121
Amount Paid: $35000.00
Employee Menu:
1. View Cars
2. View Showroom Details
3. Sell Car
4. Update Profile
5. Log Out
Enter your choice: